# EX07 - Taylor de exp(x) - Simbolos y listas

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex07_taylor_exp.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

## 1. Que queremos resolver

Construir termino a termino el polinomio de Maclaurin de e^x hasta orden N, evaluarlo en xp y compararlo con exp(xp).

### Variables / elementos del problema

| Nombre | Significado |
|---|---|
| N | orden maximo |
| x | simbolo algebraico |
| xp | punto numerico |
| ex | suma simbolica acumulada |

### Ecuacion o logica en papel

```
e^x = sum_{n=0..N} x**n / n!
```

### Comprobacion manual

Para N=2: 1+x+x**2/2. En x=0.5 da 1.625, mientras exp(0.5)~1.64872.

## 2. Lo que ya sabemos y lo nuevo

**Ya visto:**

- for
- range
- acumuladores

**Nuevo en este ejercicio:**

- SymPy
- sp.symbols()
- lista vacia y append()
- join()
- subs()
- factorial incremental

# 3. Conceptos Python nuevos

## 1. Simbolo

**Sintaxis:** `x = sp.symbols("x")`

x deja de ser un numero concreto y pasa a representar una variable algebraica dentro de una expresion.

## 2. Lista

**Sintaxis:** `terms = []`

[] crea una lista vacia que puede crecer con append().

## 3. append

**Sintaxis:** `terms.append(term)`

Añade term como un elemento nuevo al final de la lista.

## 4. join

**Sintaxis:** `" + ".join(...)`

Une una secuencia de strings usando " + " como separador.

## 5. Sustitucion

**Sintaxis:** `ex_symbolic.subs(x, xp)`

Sustituye el simbolo x por el valor xp en la expresion.

# 4. Codigo paso a paso

## Paso 1. Preparar variables

```
N = 5
x = sp.symbols("x")
xp = 0.5
ex = sp.Integer(0)
fact = 1
terms = []
```

ex es la suma acumulada, fact el factorial actual y terms guardara los terminos para mostrarlos.

## Paso 2. Iterar desde n=0

```
for n in range(0, N + 1):
```

La serie comienza con n=0.

## Paso 3. Crear y guardar el termino

```
term = x**n / fact
ex += term
terms.append(term)
```

Cada iteracion aporta x**n/n! al polinomio.

## Paso 4. Actualizar factorial

```
fact *= n + 1
```

Al hacerlo al final, fact vale 1 para el termino n=0 y queda preparado para la siguiente vuelta.

## Paso 5. Evaluar

```
ex_numeric = ex_symbolic.subs(x, xp)
```

Tras construir la expresion simbolica sustituimos x por 0.5.

## Paso 6. Comparar

```
analytical_result = exp(xp)
```

math.exp sirve como referencia numerica.

## 5. Flujo de ejecucion

```
N,xp
 -> build symbolic terms 0..N
 -> polynomial ex
 -> substitute xp
 -> compare exp(xp)
```

## 6. Resultado / comprobacion esperada

Con N=5 y xp=0.5, el archivo obtiene ~1.6486979167 frente a 1.6487212707.

## 7. Errores tipicos al empezar

- Empezar en n=1 y perder el termino 1.
- Actualizar fact antes de usarlo.
- Confundir x simbolico con xp numerico.
- Usar join sobre objetos sin convertirlos a str.

## 8. Si solo recuerdas cinco cosas

- SymPy permite algebra simbolica.
- append añade elementos a una lista.
- La serie se construye termino a termino.
- El factorial puede actualizarse incrementalmente.
- Una aproximacion se compara con una referencia.

## 9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

## Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.